

Module 5 Report: Software Assurance and Security

Emmanuel Oyeboode, JHED ID: eooyebod1

Repository: [git@github.com: eooyebode-wq/jhu_software_concepts.git](https://github.com:eooyebode-wq/jhu_software_concepts.git) (folder module_5)

1. Introduction

Software assurance is the concept of building confidence that software does what it should and nothing else. Security is one part of that work. It questions the vital idea of whether someone can make the software do something harmful. This module applies both to the Grad Cafe Flask and PostgreSQL app from Module 4. The app behaves the same as before, what changes is how it builds SQL, how it connects to the database, and how the code is checked before it ships.

A mistake made while writing code is an error. If it causes wrong behavior it escalates to a bug, and if it reaches users it is then considered a defect. Each stage costs more to fix than the one before. That is the reason this project runs its checks on every push instead of after release.

The four practices from the lecture map onto this project like this: input validation (the SQL changes and the limit clamp), static code analysis (Pylint and Snyk Code), dependency analysis (the pydeps graph and snyk test), and virtual environments (venv and uv with pinned requirements).

2. Installing and running

Both methods are run from the module_5 folder. requirements.txt lists every package with an exact version, and setup.py makes the project installable.

With pip and venv

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install -e .
```

With uv

```
uv venv
source .venv/bin/activate
uv pip sync requirements.txt
uv pip install -e .
```

I tested both from scratch in a new folder. Each one gave 10.00/10 from Pylint and 193 passing tests with 100% coverage. The uv test found a problem with my first requirements file. uv pip sync installs only what the file lists, so it skipped packages that other packages depend on, such as astroid (needed by Pylint) and pluggy (needed by pytest). pip install -r adds those on its own, which hid the problem. I replaced the short file with a complete pinned list of 68 lines, and both methods now build the same environment.

Running the app

The database settings come from five environment variables: `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER` and `DB_PASSWORD`. `.env.example` lists the names with placeholder values. A real `.env` file stays out of git.

```
cp .env.example .env      # then fill in the values
cd src
python app.py
```

The analysis page is then at <http://127.0.0.1:8080/analysis>. I confirmed that it loads when the app connects as the restricted database user described in section 6.

3. Dependency graph

The graph was made with pydeps and Graphviz from the Flask entry file:

```
pydeps src/app.py --noshow -T svg -o dependency.svg
```

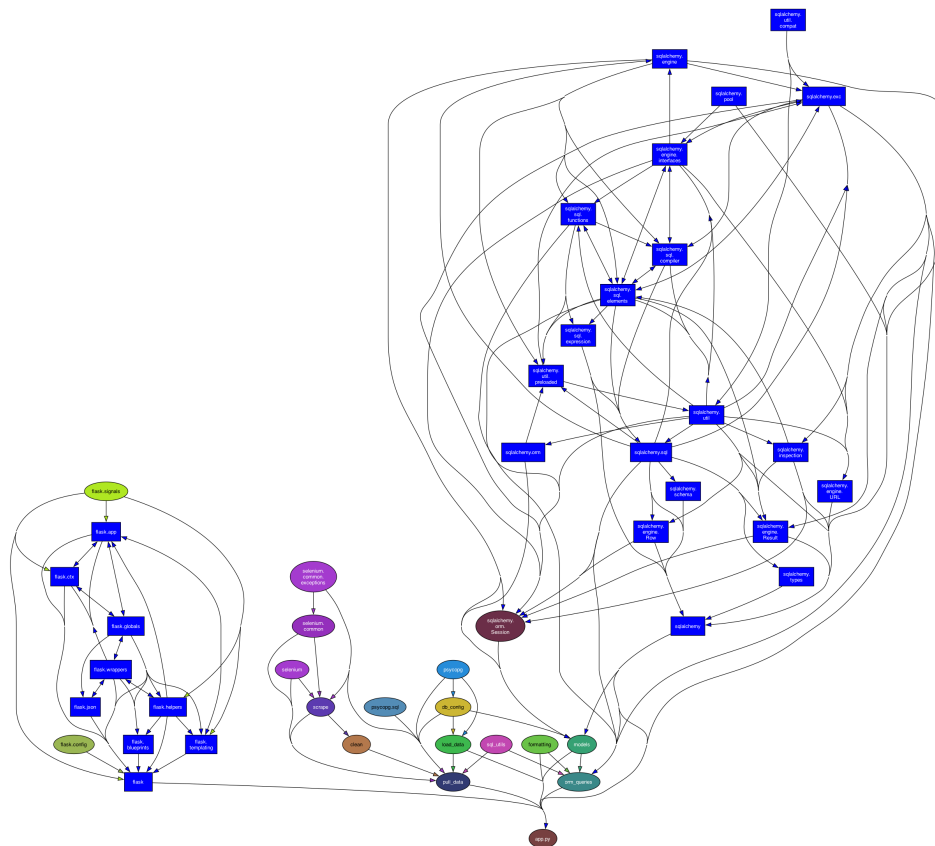


Figure 1. Dependency graph of `src/app.py` (`dependency.svg`)

The graph starts at `app.py`, the Flask entry point, which imports `orm_queries`, `models`, `load_data`, `pull_data` and `formatting`. `orm_queries` and `models` serve the analysis page: `models` open SQLAlchemy sessions using settings from `db_config`, which reads the `DB_*` environment variables, and `orm_queries` runs the queries and uses `sql_utils` to cap row limits. `load_data` and `pull_data` write new rows through `psycpg`, and `pull_data` also imports `scrape`, `clean` and `selenium` to collect newly posted entries from The Grad Cafe. The outside libraries appear as clusters: Flask serves the page, SQLAlchemy and `psycpg` talk

to PostgreSQL, and Selenium drives Chrome. `formatting` and `sql_utils` import nothing from the project, so they sit at the bottom as shared helpers. `query_data.py` does not appear because nothing imports it; it is a standalone script. Knowing exactly what the code depends on is the first step toward checking those dependencies for known vulnerabilities, which is what `snyc test` does in section 9.

4. SQL injection defenses

In the lecture's terms, raw SQL built from strings is the vulnerability, and an input such as `' or 1=1 --` is the exploit. The Module 4 queries were fixed strings with no user input, so none of them could be attacked as written. But nothing in the code stopped unsafe patterns, and nothing limited how many rows a query could return. I rewrote the queries so that the safe pattern is the only one in the code.

What changed

- **Safe composition.** Every statement is built with `sql.SQL`. Table and column names go through `sql.Identifier`, and values are `%s` placeholders.
- **Separate construction and execution.** The statement is built first as a composed object. A separate call, `cursor.execute(stmt, params)`, runs it.
- **Column allow-list.** `count_matching()` accepts a column name only if it is one of the real columns of the applicants table, and raises `ValueError` otherwise.
- **No string building.** A scan of `src/` found no SQL built with f-strings, `+`, `%` or `.format()`.
- **ORM side.** The SQLAlchemy queries already bind their values. I added a limit to each one.

Before and after

```
# The unsafe pattern, which the project does not use:
cur.execute(f"SELECT COUNT(*) FROM applicants WHERE term ILIKE '{value}'")

# What the code does now (query_data.py):
stmt = sql.SQL(
    "SELECT COUNT(*) FROM {table} WHERE {column} ILIKE %s LIMIT %s"
).format(table=APPLICANTS, column=sql.Identifier(column))
cur.execute(stmt, (value, 1))
```

Why this is safe

With binding, the database receives the SQL text and the values separately. The text `' or 1=1 --` never becomes part of the SQL. It is compared with the term column as an ordinary string, so it matches no rows. With the unsafe pattern the same text would have turned the condition into one that is always true, and every row would match.

Tests

`tests/test_sql_injection.py` sends five attack strings to `count_matching` and also uses each one as a column name. The strings are the always-true input, statement stacking, a data-changing UPDATE, a DROP TABLE, and a quote-breaking value. In every case no rows match, the table is identical afterwards, and a bad column name is refused with `ValueError`. The suite has 193 tests with 100% coverage.

5. LIMIT enforcement

- **Every query has a LIMIT.** Each statement ends in LIMIT %s. The single-row aggregates use a limit of 1. The ORM queries use .limit().
- **A maximum is enforced.** clamp_limit() in sql_utils.py turns any requested limit into a whole number from 1 to 100. Zero and negative values become 1, large values become 100, and text, None, nan and inf fall back to a default of 10.
- **Only specific exceptions are caught.** clamp_limit() catches ValueError, TypeError and OverflowError, the three ways int() can fail, and nothing broader.
- **The ID lookup is paged.** pull_data.py needs every stored p_id, so it reads them in pages of 100 instead of one unbounded query.
- **Tests.** One test checks that every raw statement contains LIMIT. Another records every statement SQLAlchemy sends while the page builds and checks the same thing. A third asks for 10**9, -1, 0, text and None rows from a table with 250 rows and checks that between 1 and 100 come back.

6. Least-privilege database configuration

The app connects as a role named gradcafe_app that has only the rights it needs. The analysis page reads the applicants table and Pull Data adds new rows, so the role can read and insert on that one table and do nothing else. The credentials are not in the code. They come from environment variables, and .env is listed in .gitignore.

Right	Why
CONNECT on the gradcafe database	The role has to be able to open the database. Other roles lose default access.
USAGE on schema public	Needed to see the table.
SELECT on applicants	The analysis page reads the data.
INSERT on applicants	Pull Data adds new rows. ON CONFLICT DO NOTHING needs only INSERT.
Not granted: UPDATE, DELETE, TRUNCATE	Nothing in the app edits or removes rows.
Not granted: ownership, CREATE	This blocks DROP, ALTER and CREATE TABLE. The table is created once by an admin account.
Not a superuser; no CREATEDB or CREATEROLE	A stolen password cannot be used to take over the server.

The SQL used (db/least_privilege.sql):

```
CREATE ROLE gradcafe_app WITH LOGIN
    NOSUPERUSER NOCREATEDB NOCREATEROLE NOREPLICATION;
REVOKE ALL ON DATABASE gradcafe FROM PUBLIC;
GRANT CONNECT ON DATABASE gradcafe TO gradcafe_app;
GRANT USAGE ON SCHEMA public TO gradcafe_app;
```

```
REVOKE CREATE ON SCHEMA public FROM gradcafe_app;
GRANT SELECT, INSERT ON TABLE applicants TO gradcafe_app;
```

The password was set inside psql with \password gradcafe_app, so it never appears in a file.

```
gradcafe=# \du gradcafe_app
List of roles
Role name | Attributes
-----+-----
gradcafe_app |
```

Figure 2. \du gradcafe_app: the role has no Superuser, Create role or Create DB attributes

```
gradcafe=# \dp applicants
Schema | Name | Type | Access privileges | Column privileges | Policies
-----+-----+-----+-----+-----+-----
public | applicants | table | emmanueloyebode=arwdDxtm/emmanueloyebode+ | | 
(1 row)
```

Figure 3. \dp applicants: gradcafe_app=ar means append (INSERT) and read (SELECT) only

```
emmanueloyebode@Emmanuel-MacBook-Air module_5 % psql -U gradcafe_app -h localhost -d gradcafe -c "DROP TABLE applicants;"
ERROR: must be owner of table applicants
```

Figure 4. A DROP TABLE attempt as gradcafe_app is refused

This is the Zero Trust idea from the lecture. Many attacks succeed by abusing an identity that has too much privilege, as in the SolarWinds case. If an injection ever got past the parameter binding, the damage would be limited to what this role can do. The lecture's example of an UPDATE that makes a user an admin would fail outright, because the role has no UPDATE right.

7. Why packaging (setup.py) matters

setup.py makes the project installable. It maps the src folder as the source of the project's modules and lists the runtime dependencies. After pip install -e . the modules can be imported from any folder, in local runs, in tests and in CI. I checked this by importing the project from the root folder of the machine.

- It removes the need for path hacks. The test setup currently adds src to sys.path by hand, which is a common cause of it works on my machine problems.
- An editable install points at the working files, so edits take effect without reinstalling.
- Tools such as uv can read the requirements from setup.py when syncing an environment.

One limit: a plain pip install . does not copy the Flask templates and static folders. They are found in an editable install, which is what the project uses.

8. GitHub Actions workflow

The workflow file is .github/workflows/ci.yml. It runs on every push and pull request, with four separate jobs that work inside module_5.

Job	What it does
pylint	Installs the pinned requirements and runs <code>pylint src --fail-under=10</code> .
dependency-graph	Installs Graphviz with <code>apt-get</code> , runs <code>pydeps</code> , and fails if <code>dependency.svg</code> is missing or empty.
snyk	Runs <code>snyk test</code> on <code>requirements.txt</code> using the <code>SNYK_TOKEN</code> repository secret. It fails if Snyk finds vulnerabilities.
pytest	Starts a Postgres 16 service container, sets the <code>DB_*</code> variables, and runs the full suite including the injection tests. <code>pytest.ini</code> requires 100% coverage.

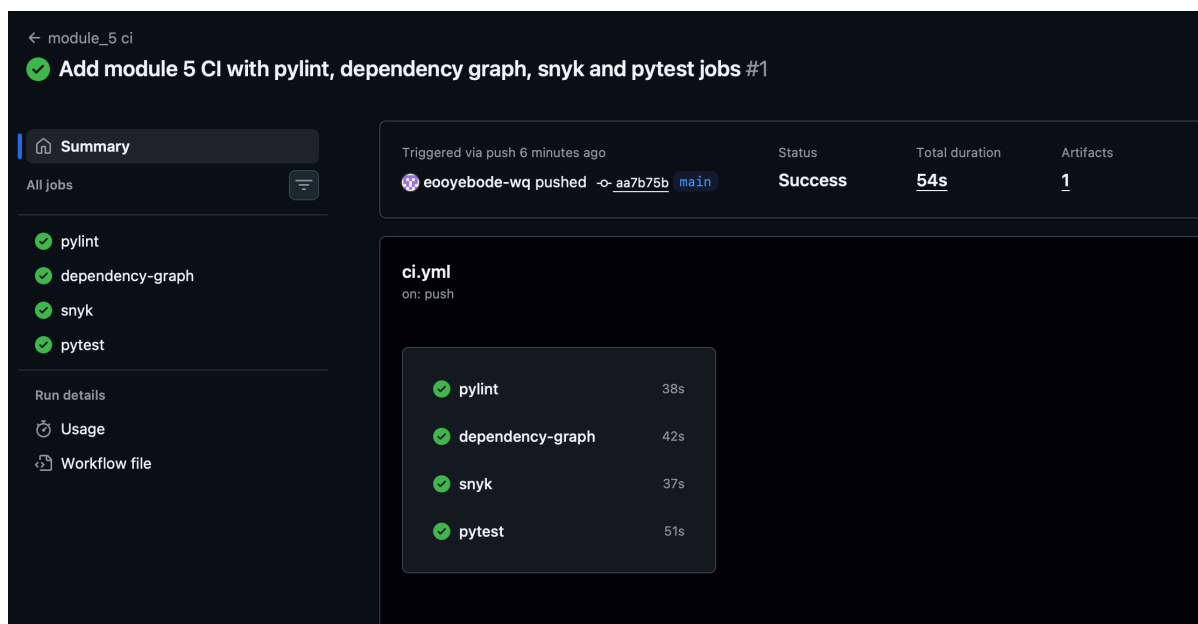


Figure 5. A successful run with all four jobs green

This is shift-left in practice. A style problem, a vulnerable dependency or a failing test is caught when it is pushed, when it is cheapest to fix.

9. Pylint and Snyk results

Pylint

Pylint is static analysis: it reads the code without running it and subtracts points for each problem it finds. The first run on `src` scored 9.73. The final run scores 10.00/10 with no warnings or errors (`pylint_report.txt`). The command is:

```
pylint src
```

The fixes changed the code instead of silencing the checks:

- Every broad `except Exception` was replaced with the specific exceptions that code can raise.
- Two functions with too many local variables were split into smaller ones.
- Four helper functions had their leading underscore removed, because other modules use them.

- The formatting helpers that were copied into two files now live in one shared module.
- The local LLM server in llm_hosting was moved out of src, because it is not part of the Flask app and needs packages that are not installed here.

Two inline disables remain, both too-few-public-methods on the SQLAlchemy model classes in models.py. Those classes only describe table columns. Each has a short comment giving the reason.

Snyk Open Source (snyk test)

```
snyk test --file=requirements.txt --package-manager=pip
```

```
Testing (requirements.txt) ...
✓ Tested 62 dependencies for known issues, no vulnerable paths found.

Test Summary
Organization: eooyebode-wq
Test type: open-source
Project path: /Users/emmanueloyebode/Documents/SCHOOL/MASTERS/Modern Software Concepts /jhu_software_concepts/module_5
Total issues: 0

Tip
To view ignored issues, use the --include-ignores option.
```

Figure 6. snyk test: 62 dependencies tested, no vulnerable paths found

Snyk tested 62 dependencies against its vulnerability database and found no issues, so nothing had to be upgraded or removed. Exact version pins in requirements.txt also mean a package cannot change underneath the project, which is the lesson of the PyPI hijacking example.

Snyk Code (snyk code test), extra credit

snyk code test src found three low-severity issues, all the same type (output saved in snyk-code-analysis.txt).

File	Line	Finding
clean.py	88	Path traversal: a command-line argument is used as the path in save_data.
clean.py	93	Path traversal: a command-line argument is used as the path in load_data.
load_data.py	151	Path traversal: a command-line argument is used as the path in read_records.

I accepted these three. The paths are typed on the command line by the person running the script, who already has the same file access. No outside user supplies them, and the Flask app never passes user input to these functions. If these functions were ever called from a web route, the paths would need to be restricted to a fixed folder.

10. SQL requirements checklist

Requirement	How it is met
No SQL built from strings	No f-strings, +, % or .format() on SQL text in src/.
Safe composition	sql.SQL with sql.Identifier for names.
Values parameterized	All values are %s placeholders with a separate params tuple.
Construction separate from execution	Statements are composed objects, run with cursor.execute(stmt, params).
LIMIT on every query	Every statement ends in LIMIT %s, and the ORM queries use .limit().
Maximum enforced	clamp_limit keeps limits between 1 and 100.
Malicious input handled	Five attack strings tested: no crash, no extra rows, no data changed.
Least privilege	gradcafe_app has SELECT and INSERT on applicants only.